

FrameLabs

Product Requirements Document

Version 1.0 | Product design baseline | 29 September 2026

A collaborative workspace for developers to create, edit, review, and preserve engineering diagrams and technical documentation in one project.

Status: Draft for product review

1. Product overview

FrameLabs is a project-based workspace for developers and engineering teams. A project contains multiple independent canvases, each optimized for a specific engineering artifact: Markdown notes, database schemas, UML diagrams, architecture diagrams, or process flows.

The product combines familiar established syntaxes with interactive visual editors. Users can work alone or collaborate in real time, while project-level permissions control who can view, edit, save, and manage content.

Success means a developer can open a project, create the right canvas for a design problem, collaborate with teammates, and return later to a clear saved version without stitching together several unrelated tools.

1.1 Product principles

- Use established formats where they improve portability: Markdown, DBML, PlantUML, and Mermaid.
- Keep each canvas independent. There are no cross-canvas links or backlinks in this product version.
- Make visual editing fast and understandable; syntax is an optional representation unless the canvas explicitly uses it as its source of truth.
- Treat collaboration, permissions, and saved history as core product behavior rather than add-ons.
- Prefer a shared editor foundation over duplicated canvas infrastructure.

1.2 Target users

User	Primary needs
Software developers	Design systems, databases, workflows, and documentation quickly.
Engineering teams	Collaborate on diagrams, review changes, and maintain a shared design record.
Technical leads / architects	Communicate system boundaries, dependencies, and trade-offs clearly.

2. Scope

In scope for this PRD: the complete product experience from authentication through projects, canvases, collaboration, permissions, persistence, and saved history.

Explicitly excluded: deployment and hosting architecture, custom application encryption, end-to-end encryption, AI generation, cross-canvas linking, and future advanced integrations.

3. Information architecture

Projects are the top-level organizational unit. A project contains any number of independent canvases, including multiple canvases of the same type.

3.1 Project hierarchy

Project > Canvas type > Canvas

Group	Canvas	Purpose
Writing	Markdown Canvas	Technical notes and documentation.
Database	DBML Canvas	Database tables, fields, keys, and relationships.
UML	Class Diagram Canvas	Classes, properties, methods, and relationships.
UML	Sequence Diagram Canvas	Actors, lifelines, messages, and interactions.
Architecture	Architecture Canvas	Services, infrastructure, boundaries, and dependencies.
Flow	Flow Canvas	Processes, decisions, branches, loops, and workflows.

3.2 Project and canvas actions

- Create, rename, duplicate, archive, and delete projects according to permissions.
- Create, rename, duplicate, archive, and delete canvases according to permissions.
- Create multiple canvases of the same type inside one project.
- Open a project and see its canvases grouped by type.
- Never imply that canvases are linked; each canvas has its own content, collaboration room, and history.

4. Canvas requirements

Canvas	Editor / renderer	Syntax or model	Source of truth
Markdown	CodeMirror 6 + Markdown preview	Markdown	Raw Markdown
DBML	CodeMirror 6 + interactive graph editor	DBML	DBML text; layout stored separately
UML class	Interactive graph editor + PlantUML adapter	PlantUML	PlantUML text
UML sequence	Timeline editor + PlantUML adapter	PlantUML	PlantUML text
Architecture	Interactive graph / visual canvas	FrameLabs visual model	Visual JSON model
Flow	CodeMirror 6 + interactive graph editor	Mermaid flowchart	Mermaid text; layout stored separately

All graph-based canvases should share selection, zoom, pan, undo/redo, node and edge manipulation, keyboard shortcuts, and collaboration behavior. Their palettes and validation rules remain different.

4.1 Markdown Canvas

- Provide a distraction-free editor with an Obsidian-like feel.
- Keep raw Markdown as the source of truth.
- Show a live rendered preview at all times, with comfortable editing and preview layouts.
- Support standard Markdown structure, links, lists, code blocks, tables, and task lists.

4.2 DBML Canvas

- Provide a DBML editor and a dbdiagram-like ERD visual view.
- Parse DBML into tables, fields, keys, indexes, and relationships.
- Allow users to move tables and inspect relationship highlighting on hover.
- Keep layout positions separate from DBML so visual rearrangement does not rewrite schema text.
- Validate malformed DBML and show actionable errors without losing the last valid render.

4.3 UML Canvas

- Support multiple UML canvases in a project.
- Provide Class Diagram and Sequence Diagram as child types.
- Use PlantUML as the established syntax and provide a visual representation for editing and review.
- Class diagrams use a graph editor; sequence diagrams use a timeline/lifeline editor.
- Visual changes must update the PlantUML representation while preserving valid content.

4.4 Architecture Canvas

- Be visual-first with no required code editor.
- Provide a palette for cloud and technology logos, generic boxes, boundaries, text, arrows, connectors, and notes.
- Support grouping, nesting, labeled connections, directional arrows, and system boundaries.
- Store the diagram as a FrameLabs visual model that is independent of Mermaid or PlantUML.
- Bundle approved icon assets locally and retain source and license metadata for each icon.

4.5 Flow Canvas

- Support actions, decisions, branches, loops, start/end states, labeled edges, and annotations.
- Use Mermaid flowchart syntax as the source of truth.
- Provide both a Mermaid code editor and an interactive visual graph editor.
- Visual edits must serialize back to valid Mermaid where supported.
- Keep layout positions separate from Mermaid text when Mermaid's automatic layout cannot preserve manual positioning.

5. Authentication and authorization

FrameLabs uses Better Auth for account authentication, sessions, password login, OAuth, and session lifecycle. Project membership and canvas permissions are owned by FrameLabs.

5.1 Roles

Role	Permissions
Owner	Full project control, membership management, canvas management, editing, saving, restoring, and export.
Editor	View and edit project canvases, collaborate, save versions, and export where enabled.
Viewer	View canvases and saved versions. Cannot edit, save, restore, or manage members.

5.2 Authorization requirements

- The server must enforce project membership on project fetch, canvas fetch, collaboration subscription, updates, saves, exports, and restores.
- The client must reflect permissions in the UI but must never be the only authorization layer.
- Removing a collaborator must revoke future access and close or invalidate active collaboration connections.
- Previously viewed or downloaded content cannot be reclaimed from a client device.

6. Collaboration and synchronization

Each canvas is an independent Yjs document. A collaboration room is scoped to one canvas, not the entire project.

- Clients connect to the canvas collaboration room through an authenticated WebSocket connection.
- The server authenticates the session, checks project membership and role, then accepts or rejects the connection.
- Yjs updates are broadcast to authorized collaborators and persisted by the server.
- Editors can submit updates; viewers receive updates but cannot submit mutations.
- Offline edits are persisted locally and synchronized when the client reconnects.
- Yjs handles concurrent document updates and merging; application authorization remains a server responsibility.

Encryption is intentionally deferred. The current product trusts the authenticated backend and uses transport security and access control as the initial security boundary.

7. Saving and version history

Live collaboration updates maintain the current draft. A permanent history version is created only when an authorized user clicks Save.

- Do not create a permanent version for every keystroke, drag, or cursor movement.
- Each saved version belongs to exactly one canvas.
- Store the saved document state plus metadata: author, timestamp, version number, and optional message.
- Allow authorized users to browse saved versions and restore a previous version.
- Restoring a version creates a new current state and should itself be recorded as a new saved version.

8. Data model (conceptual)

Entity	Important fields / relationship
User	Managed by Better Auth; identity, session, and account data.
Project	id, name, ownerId, timestamps, archived state.
ProjectMember	projectId, userId, role, status, timestamps.
Canvas	id, projectId, name, type, subtype, current document state, layout state.
CanvasVersion	id, canvasId, version number, saved state, authorId, message, timestamp.
CanvasAsset	id, canvasId, asset type, metadata, storage reference where applicable.

Canvas type determines the editor and validation rules. Canvas subtype distinguishes UML class and sequence diagrams. Canvas history is never shared between canvases.

9. Functional requirements

ID	Requirement	Priority
FR-01	A user can register, sign in, sign out, and authenticate through supported Better Auth providers.	Must
FR-02	An authorized user can create and manage projects.	Must
FR-03	An owner can invite, change roles, and remove project collaborators.	Must
FR-04	A project can contain multiple independent canvases of any supported type.	Must
FR-05	Users can create, rename, duplicate, archive, and delete canvases according to role.	Must
FR-06	Each canvas opens in its type-specific editor with the correct syntax, palette, and validation behavior.	Must
FR-07	Authorized collaborators can edit a canvas concurrently and see updates in near real time.	Must
FR-08	An authorized user can explicitly Save a canvas to create a permanent version.	Must
FR-09	Authorized users can browse and restore saved versions.	Must
FR-10	The server rejects unauthorized project, canvas, collaboration, update, save, and restore requests.	Must
FR-11	Flow and UML syntax errors are shown without destroying the last valid visual state.	Should
FR-12	Architecture icons can be searched and inserted from the local icon registry.	Should

10. Non-functional requirements

- Responsive editing: ordinary pointer and text actions should feel immediate during local editing.
- Resilience: reconnecting clients must recover the latest authorized document state without data loss.
- Consistency: all clients in a canvas converge to the same Yjs document state.
- Security: authorization is checked server-side at every resource and mutation boundary.
- Accessibility: keyboard navigation, visible focus, readable contrast, and non-color-only status indicators.
- Maintainability: graph primitives and collaboration infrastructure are shared across graph-based canvases.
- Portability: syntax-backed canvases retain valid Markdown, DBML, PlantUML, or Mermaid source.

11. Primary user flows

11.1 Create and edit a canvas

Sign in -> open or create a project -> choose canvas type -> name the canvas -> edit locally -> collaborate if connected -> click Save to create a version.

11.2 Collaborate

Editor opens a canvas -> client authenticates the WebSocket room -> server checks membership and role -> Yjs updates are exchanged -> all authorized clients converge -> an editor clicks Save when the team wants a permanent version.

11.3 Revoke access

Owner removes or changes a member -> server invalidates the membership -> active collaboration connection is closed or denied -> future project and canvas requests return an authorization error.

12. Acceptance criteria

- A developer can create a project containing at least one canvas of every supported type.
- Two authorized editors can modify the same canvas and observe synchronized changes.
- A viewer cannot mutate content through the UI or by sending a crafted request.
- A user can edit Markdown, DBML, PlantUML, and Mermaid content while preserving valid source.
- A user can create a visual architecture diagram with boxes, arrows, boundaries, and registered icons.
- Dragging DBML or Flow nodes changes layout without unexpectedly rewriting source syntax.
- Clicking Save creates one permanent version; ordinary live edits do not create endless history entries.
- Restoring a version changes the current canvas state and creates an auditable new saved version.
- Removing a collaborator prevents future reads and edits and ends active collaboration access.

13. Deferred decisions and future scope

Deferred item	Reason
End-to-end encryption	Adds key management and makes server-side search, validation, rendering, and RBAC workflows harder.
Custom encryption and KMS	Not required for the initial trusted-backend product boundary.
Deployment architecture	Explicitly excluded from this PRD.
Cross-canvas links	Canvases are intentionally independent in this product version.
AI diagram generation	Not required to validate the core editor and collaboration experience.
Advanced presence and comments	Can be added after core synchronization and permissions are reliable.

14. Product decision summary

FrameLabs will be a collaborative, project-based engineering workspace with independent Markdown, DBML, UML, Architecture, and Flow canvases. Syntax-backed canvases preserve established formats; Architecture uses a FrameLabs visual model. Yjs provides real-time document synchronization, Better Auth provides identity and sessions, FrameLabs enforces project-level RBAC, and explicit Save creates permanent history versions.

This PRD is ready for product review. Implementation planning should begin only after the product scope and canvas behavior are approved.

Appendix A: Two-Developer Delivery Phases

Phases describe build order, not reduced product scope. Each phase ends with an integration checkpoint.

Phase	Backend developer	Frontend developer	Integration result
0. Shared contract	Define schemas, APIs, canvas document formats, roles, and errors.	Define routes, shared canvas interfaces, UI patterns, and editor boundaries.	Both developers work from the same contracts.
1. Product foundation	Better Auth, users, projects, members, and RBAC APIs.	App shell, login, project list, project page, and canvas sidebar.	A user can sign in, create a project, and see canvases.
2. Shared canvas platform	Canvas CRUD, draft persistence, and permission middleware.	Canvas shell, toolbar, zoom/pan, selection, undo/redo, and states.	Any canvas type can plug into the shared editor.
3. Markdown vertical slice	Markdown storage and Save/version APIs.	CodeMirror, live preview, and Obsidian-like layout.	First complete end-to-end canvas.
4. Collaboration	Yjs WebSocket server, room authorization, and update persistence.	Yjs provider, remote updates, presence, connection state, and offline queue.	Two users can edit one Markdown canvas together.
5. DBML Canvas	Storage support, payload validation, and version integration.	DBML editor, parser, ERD renderer, table movement, and relationship highlighting.	Collaborative database diagrams with history.
6. Architecture Canvas	Asset metadata APIs and project permission integration.	Visual graph editor, logos, boxes, boundaries, arrows, labels, and grouping.	End-to-end architecture diagrams.
7. UML Canvas	Syntax/document persistence and validation boundaries.	Class editor, sequence timeline, and PlantUML adapter.	Class and sequence diagrams.
8. Flow Canvas	Mermaid document persistence and validation boundary.	Mermaid editor, visual flow editor, parser/serializer, decisions, and branches.	Mermaid-backed flow diagrams.
9. Hardening	Security review, revocation, recovery, and query optimization.	Accessibility, performance, keyboard shortcuts, and edge-case UX.	Product-wide quality pass.

Ownership boundary

Backend owns authentication, project membership, RBAC, persistence, version history, WebSocket authorization, and Yjs update handling. Frontend owns rendering, syntax editors, visual interactions, parsing/serialization adapters, layout, and user-facing validation. The backend stores and validates document payloads but does not duplicate every diagram renderer.

Critical milestone

Phase 3 creates the first complete vertical slice. Phase 4 validates collaboration on that slice before the more complex diagram editors are built.

Appendix B: Technology Stack

The stack favors familiar TypeScript tools, a shared graph editor foundation, and established diagram syntaxes.

Area	Technology	Purpose
Frontend	React + TypeScript + Vite	Interactive application and canvas UI.
Styling	Tailwind CSS	Consistent product interface styling.
Diagram engine	React Flow	Nodes, edges, zoom, pan, selection, and custom diagram interactions.
Text editor	CodeMirror 6	Markdown, DBML, Mermaid, and PlantUML editing.
UI icons	Lucide React	Toolbar and general interface icons.
Markdown	remark / remark-gfm	Markdown parsing and live preview.
Database diagrams	DBML parser + React Flow	Schema parsing and interactive ERD rendering.
Flow diagrams	Mermaid	Flowchart syntax, rendering, and source representation.
UML diagrams	PlantUML adapter	Class and sequence diagram syntax.
Backend	Node.js + TypeScript + Express	REST APIs and application server.
Validation	Zod	Request and shared document payload validation.
Database	PostgreSQL + Drizzle ORM	Users, projects, memberships, canvases, and versions.
Authentication	Better Auth	Password login, OAuth, sessions, and account lifecycle.
Collaboration	Yjs + WebSocket	Concurrent editing and real-time synchronization.
Offline storage	IndexedDB	Local drafts and queued collaboration updates.
Optional coordination	Redis	Presence, pub/sub, and multi-instance coordination when needed.
Local services	Docker Compose	Repeatable local PostgreSQL and Redis services.

Repository structure

apps/web contains the React frontend. apps/server contains the Express API and WebSocket server. packages/shared contains types, Zod schemas, and API contracts. packages/canvas-core contains shared canvas behavior, while packages/editors contains canvas-specific adapters.

Backend boundary

The backend owns authentication, authorization, persistence, version history, and synchronization. The frontend owns rendering, syntax parsing and serialization, visual interactions, and editor-specific behavior.

Deliberate simplification

Redis remains optional until multi-instance collaboration or presence requires it. Custom encryption, KMS, and deployment tooling are outside this PRD.